SDET Automation Interview Questions

Based on your Java Playwright + REST Assured framework

Project: Playwright_FrameWork
Core stack: Java 21, Playwright Java, TestNG, REST Assured, Allure, Maven, GitHub Actions, Jenkins
Focus file: src/test/java/com/qa/opencart/tests/Users_Data_API.java

Use this as a practical technical-round revision document. The best interview answers should cover what you built, why you chose it, how it works internally, and what you would improve next.

1. Your 60-Second Project Pitch

Question: Explain your framework architecture.

Answer direction:

This is a Java Playwright automation framework for the OpenCart application. UI tests follow Page Object Model using `HomePage` and `LoginPage`. `BaseTest` handles browser setup and teardown, while `PlaywrightFactory` creates Playwright, Browser, BrowserContext, and Page. For API testing, `BaseApiTest` loads config without launching a browser, and `Users_Data_API` uses REST Assured for CRUD validation against GoRest APIs. Test execution is controlled by TestNG XML suites, reports are generated with Allure, screenshots and failure details are attached through a TestNG listener, and CI is configured through GitHub Actions and Jenkins.

2. Framework Architecture

1. Why did you use Page Object Model?

To keep locators and page actions separate from test assertions. For example, `HomePage.doSearch()` hides search implementation from `HomePageTest`, making tests cleaner and easier to maintain.

2. What is the role of `BaseTest`?

It centralizes UI setup: load config, launch browser, create page objects, and close browser context after execution.

3. Why did you create `BaseApiTest` separately?

API tests do not need a browser. `BaseApiTest` only loads configuration and avoids wasting resources by launching Playwright.

4. What is the responsibility of `PlaywrightFactory`?

It initializes Playwright, browser, context, and page based on config. It also supports browser selection, headless mode, slow motion, grid flags, and ThreadLocal isolation.

5. Why is framework configuration externalized?

Browser, URL, credentials, and API token can change by environment. Keeping them in properties or system variables avoids code changes and keeps secrets out of source code.

6. What is one design improvement you would make?

Replace hardcoded API base URLs with config constants, introduce request/response POJOs, and use a JSON serializer instead of string concatenation in `Payload`.

3. Playwright Java

1. Why Playwright instead of Selenium?

Playwright has auto-waiting, browser context isolation, strong network/API capabilities, modern browser support, and generally less flaky synchronization.

2. What is the difference between Browser, BrowserContext, and Page?

Browser is the running browser instance. BrowserContext is an isolated session with separate cookies/storage. Page is a tab inside that context.

3. Why do you close `page.context()` in teardown?

Closing the context clears session state and releases page resources. It is lighter than closing the full Playwright engine after every test block.

4. How does Playwright reduce flaky tests?

It auto-waits for elements to be actionable before actions like click/fill. Still, explicit waits such as `waitForURL()` are useful for navigation success checks.

5. Why did login validation use URL wait instead of only checking Logout visibility?

The Logout link may be hidden in a dropdown, so visibility can be unreliable. The account dashboard URL is a stronger success signal.

6. What locator strategy did you use and what would you improve?

This project uses XPath/CSS strings. I would prefer stable role-based locators, test IDs, or semantic selectors where the app supports them.

4. Java Concepts Used

1. Where did you use inheritance?

Test classes extend `BaseTest` or `BaseApiTest` to reuse setup logic.

2. Where did you use encapsulation?

Page classes keep locators private and expose actions like `doSearch()` and `doLogin()`.

3. Where did you use static members?

`AppConstants` stores common expected values, `Payload` has static payload helper methods, and `PlaywrightFactory` exposes static ThreadLocal getters.

4. Why use `ThreadLocal` in `PlaywrightFactory`?

In parallel execution, each thread needs its own Page, Browser, BrowserContext, and Playwright instance. ThreadLocal prevents one test thread from overwriting another test's browser state.

5. Why use `ConcurrentHashMap` in `Payload`?

Created user IDs are stored by email. Since tests may run in parallel, `ConcurrentHashMap` is safer than `HashMap` for concurrent access.

6. What is a risk in `Payload`?

It still has static fields like `createdUserName` and `createdUserEmail`. In parallel execution, those can be overwritten. A better design is POJO-based test data passed through the test flow.

7. Why is string concatenation for JSON risky?

It can break with quotes, special characters, or escaping issues. Jackson ObjectMapper or POJO serialization would be safer.

8. Where is exception handling used?

Config loading catches file errors, invalid browsers throw `IllegalArgumentException`, and login catches `PlaywrightException` after a timeout.

5. TestNG

1. Why TestNG?

It supports annotations, priorities, dependencies, DataProviders, XML suite control, listeners, groups, and parallel execution.

2. How does `@DataProvider` work in your framework?

It supplies multiple sets of data to the same test method, such as product search terms in `HomePageTest` and user data in `Users_Data_API`.

3. What is the risk of using `priority` and `dependsOnMethods`?

Tests become order-dependent. If create fails, get/update/delete are skipped. It is acceptable for CRUD flow testing, but independent tests are better for regression stability.

4. What does `parallel="tests"` mean in `testng_regression.xml`?

TestNG runs separate `<test>` blocks in parallel. In this project, Home, Login, and API suites can run concurrently.

5. How does your listener work?

`TestAllureListners` implements `ITestListener`. It captures screenshots on failure/skip, attaches logs, sets Allure history IDs, and creates a run summary.

6. What is `IRetryAnalyzer`?

It reruns failed tests up to a configured limit. In this project, `Retry` allows up to three retries, useful for transient failures.

6. REST Assured And API Testing

1. Explain the CRUD flow in `Users_Data_API`.

The test creates users with unique data, stores the generated ID, lists users, gets the created user, updates it, verifies updated values, deletes it, and confirms a 404 after deletion.

2. Why did you use `System.currentTimeMillis()`?

To create unique names and emails so repeated runs do not fail because of duplicate email constraints.

3. How do you handle authentication?

The GoRest bearer token is read from config/system property, then passed using `given().auth().oauth2(token)`.

4. What assertions do you perform on create user?

Status code 201, response time, headers like Content-Type and Location, security/rate-limit headers, ID type/value, and body fields such as name, email, gender, and status.

5. Why validate headers, not only body?

Headers are part of the API contract. Content-Type, Location, cache, security, and rate-limit headers can reveal integration or security regressions.

6. What is the difference between PUT and PATCH?

PUT usually replaces a resource, PATCH partially updates it. In this project the method name says PATCH but the request uses `put()`, so that is a good correction to make.

7. Why verify 404 after DELETE?

A 204 only says the delete request was accepted. A follow-up GET confirms the resource is no longer available.

8. What is JsonPath used for?

To extract and assert specific fields from JSON responses. The mock response test reads array size, pagination, title, and price fields.

9. What is Hamcrest used for?

Fluent assertions such as `lessThan`, `containsString`, `matchesPattern`, `everyItem`, `notNullValue`, and `equalTo`.

10. How would you improve API test maintainability?

Create `RequestSpecification` and `ResponseSpecification`, add POJOs, move base URI to config, add JSON schema validation, and add negative tests for 400/401/422/429.

7. Allure, Reporting, And AI-Era Quality

1. What does Allure add to your framework?

Rich reports with severity, owner, descriptions, attachments, screenshots, and cross-run trends.

2. How are screenshots captured?

On failure or skip, the TestNG listener gets the current thread's Page from `PlaywrightFactory.getPage()` and attaches `page.screenshot()` to Allure.

3. What does `FailureClassifier` do?

It extracts the failed test name and throwable message into a readable failure summary for the Allure report.

4. How does flaky detection work?

`AllureHistoryReader` reads historical test statuses, and `FlakyDetector` marks a test flaky if it has at least three runs and both passed and failed statuses.

5. Is this true AI?

It is not an LLM-based system yet. It is rule-based failure classification and flaky detection. In interview, call it "AI-ready observability" or "rule-based failure analytics", not full AI.

6. How would you make this AI-era ready?

Feed Allure failures, screenshots, logs, traces, and API responses into an LLM summarizer to classify root cause as product bug, test data issue, locator issue, environment issue, or flaky timing issue.

7. What AI use cases are realistic for SDET?

Test case generation from requirements, API contract test generation from OpenAPI specs, failure clustering, flaky test prediction, log summarization, test impact analysis, and self-healing locator suggestions with human review.

8. CI/CD

1. How does GitHub Actions run the framework?

It checks out code, sets up Java, installs dependencies, installs Playwright browsers, and runs Maven tests headlessly with credentials and token from secrets.

2. Why run headless in CI?

CI agents usually do not have a display. Headless mode makes browser tests reliable in server environments.

3. What is Maven Surefire doing?

Surefire runs the TestNG suite configured in `pom.xml` and produces test reports under `target/surefire-reports`.

4. How does Jenkins differ from GitHub Actions here?

Jenkinsfile defines stages like build, deploy to QA, regression automation, and publish Allure report. GitHub Actions is simpler and tied to repository events.

5. What would you improve in CI?

Persist Allure history, upload screenshots/reports as artifacts, split UI and API jobs, run API tests on every PR, run full UI regression nightly, and fail build based on quality gates.

9. Senior-Level Improvement Questions

1. How would you remove test order dependency from the API suite?

Make each test create and clean up its own data, or use a dedicated CRUD scenario test while keeping other tests independent.

2. How would you support multiple environments?

Use environment-specific config files or system properties like `-Denv=qa`, then load URLs, credentials, and API base URLs based on that value.

3. How would you add contract testing?

Add Pact or schema validation to verify API contracts independently of full end-to-end flows.

4. How would you reduce flaky UI failures?

Use stable locators, remove hard waits, use Playwright auto-waiting and explicit URL/state waits, isolate test data, and avoid shared static state.

5. How would you design API negative tests?

Cover missing token 401, invalid token 401/403, duplicate email 422, invalid payload 422/400, deleted user 404, unsupported method 405, and rate limit 429.

6. What security practices does this project show?

It avoids committing real credentials by supporting system property overrides and GitHub secrets.

7. What is the biggest technical debt in the current framework?

API payload construction and state handling. JSON should use POJOs/ObjectMapper, and static shared state should be minimized for parallel-safe testing.

10. Rapid-Fire Questions Recruiters Can Ask

1. What is POM?

2. What is ThreadLocal and why is it used here?

3. Difference between `@BeforeTest`, `@BeforeClass`, and `@BeforeMethod`?

4. Difference between PUT and PATCH?

5. Difference between 200, 201, 204, 400, 401, 403, 404, 422, and 500?

6. How do you pass secrets in CI?

7. What is the difference between API testing and UI testing?

8. What is JSON schema validation?

9. What are query params and path params?

10. How do you validate response time?

11. What is a flaky test?

12. How do retries help and how can they hide bugs?

13. What are Allure attachments?

14. What are TestNG listeners?

15. Why should API tests not launch a browser?

16. Why is `ConcurrentHashMap` better than `HashMap` in parallel execution?

17. What is serialization/deserialization?

18. What is ObjectMapper?

19. What would you do if login passes locally but fails in CI?

20. How would you debug an intermittent API 429 failure?

11. Interview-Safe Improvement Statement

Use this when asked "what would you improve in your framework?"

I would improve the framework in four areas. First, I would move REST Assured common setup into reusable request and response specifications. Second, I would replace string-based JSON payloads with POJOs and Jackson serialization. Third, I would reduce static shared state in API tests to make parallel execution safer. Fourth, I would expand reporting by preserving Allure history in CI and using failure logs/screenshots/API responses for AI-assisted failure classification.

Last-Minute Revision Checklist

1. **Be ready to explain the full API CRUD chain: POST, GET, PUT/PATCH, GET updated, DELETE, and 404 verification.**
2. **Mention that API tests extend BaseApiTest so they do not launch a browser.**
3. **Explain ThreadLocal clearly: one Playwright Page/Context per test thread for parallel safety.**
4. **Admit the right improvements: POJOs, ObjectMapper serialization, RequestSpecification, ResponseSpecification, JSON schema validation, and less static shared state.**
5. **For AI-era SDET questions, talk about failure clustering, flaky prediction, log summarization, test impact analysis, and human-reviewed self-healing locators.**
6. **If asked about a weakness, use the PUT/PATCH mismatch and string-based JSON payloads as honest improvement examples.**

> **Strong closing line:** I built this as a hybrid UI and API automation framework, and the next step I would take is making the API layer more contract-driven, strongly typed, and CI-observable with better historical reporting.